

Assignment 1: Testing and evaluating the BRICS Java Automata Library, as a case study for JUnit5, Combinatorial Interaction Testing, and Mutation testing

Topics in Software Engineering

Suggested duration: 4 weeks

Required technologies:

- Java
 - JUnit5
 - PIT
 - PICT
 - Maven or Gradle
 - Git
 - Static HTML
-

1. Overview

In this assignment, you will conduct an empirical software engineering study on the BRICS Java automata library: <https://github.com/cs-au-dk/dk.brics.automaton>

The BRICS automata library is a well-known open-source Java library implementing:

- finite automata
- regular expressions
- automata transformations
- determinization
- minimization
- intersection
- union
- complement
- language membership operations

This assignment has two central goals:

1. Evaluating the quality, correctness, and software design of the BRICS automata library
2. Evaluating the effectiveness of combinatorial interaction testing (CIT) using PICT and BRICS as a case study

You will investigate how well combinatorial interaction testing contributes to:

- coverage
- mutation score
- fault detection
- metamorphic-property validation
- exploration of complex input spaces

You will also analyze the software design and implementation quality of the BRICS library itself.

This assignment combines:

- software testing
 - empirical software engineering
 - combinatorial testing
 - metamorphic testing
 - mutation testing
 - software design analysis
 - reproducibility engineering
-

2. Main Goals

Goal 1 — Evaluating the BRICS Library

You will evaluate the BRICS automata library in terms of:

- correctness
- testability
- mutation resistance
- software design quality
- adherence to software engineering principles

This includes:

- generating a JUnit5 test suite
 - mutation testing using PIT
 - metamorphic testing
 - differential testing against `java.util.regex`
 - software-design evaluation
 - refactoring analysis
-

Goal 2 — Evaluating the Effectiveness of Combinatorial Interaction Testing

You will evaluate combinatorial interaction testing (CIT) using PICT and BRICS as a case study.

You will investigate questions such as:

- How effective is 3-wise coverage for this domain?
- Does CIT improve mutation scores?
- Does CIT expose behaviors missed by tests written by other methods?
- How large are the generated test suites?
- What are the tradeoffs between coverage strength and suite size?
- Which types of faults or behaviors are best exposed using CIT?

- Does increasing the interaction strength improve fault detection?
- At what cost does higher interaction strength increase the number of generated test inputs?

You should empirically compare different interaction strengths where feasible, for example:

- 2-wise
- 3-wise
- 4-wise

and evaluate both:

- mutation scores
- generated test-suite sizes

3. Research Questions

RQ1 — Effectiveness of the Generated Test Suite

How effective is your JUnit5 test suite for the BRICS automata library?

You should evaluate:

- line coverage
- branch coverage
- mutation score using PIT
- fault detection capability
- readability and maintainability

RQ2 — Effectiveness of Combinatorial Interaction Testing

How effective is combinatorial interaction testing for testing the BRICS automata library?

You should evaluate:

- effectiveness of 2-wise, 3-wise, and 4-wise coverage
- mutation-score improvements from CIT-generated inputs
- additional behaviors explored by CIT
- tradeoffs between coverage strength and test-suite size
- limitations of combinatorial testing
- whether increasing interaction strength improves fault detection
- how interaction strength affects the number of generated test inputs

You should compare multiple interaction strengths using metrics such as:

- mutation score
- number of generated test cases

- execution time
 - additional mutants killed
-

RQ3 — Correctness Evaluation

Does the library satisfy expected semantic properties?

You should evaluate correctness using:

- metamorphic testing
 - differential testing against `java.util.regex`
 - combinatorial interaction testing
 - oracle-based checks where available
-

RQ4 — Software Design Evaluation

What software-design strengths and weaknesses exist in the BRICS automata library?

You should evaluate:

- adherence to the SOLID principles
- use of design patterns
- maintainability
- extensibility
- API clarity
- complexity
- duplication
- documentation quality
- testability

You should also identify:

- concrete refactoring opportunities
 - architectural weaknesses
 - classes or methods violating good OO design principles
 - tradeoffs involved in possible refactorings
-

4. Required Technologies

Programming language: Java
Unit testing: JUnit5
Mutation testing: PIT
Coverage measurement: JaCoCo or equivalent
Combinatorial testing: PICT

Build system: Maven or Gradle
Version control: Git
Website: Static HTML

5. Repository Setup

You must fork or clone the BRICS repository.

You must document:

- commit hash used
 - Java version
 - operating system
 - build commands
 - dependencies
 - modifications made
-

6. Generating a New JUnit5 Test Suite

You must generate a new JUnit5 test suite for the complete library.

The new test suite may include:

- manually written tests
- automatically generated tests, e.g., AI-assisted tests
- parameterized tests
- metamorphic tests

The generated suite must:

- use JUnit5
 - compile automatically
 - run automatically
 - integrate with Maven or Gradle
 - be clearly separated from the library source code
-

7. Evaluating the Generated Test Suite

You must evaluate the quality of your generated JUnit5 test suite.

At minimum, measure:

- line coverage
- branch coverage
- mutation score using PIT

- number of tests
- number of assertions
- test execution time

You should also discuss:

- readability
 - maintainability
 - flaky behavior
 - usefulness of assertions
 - ease of debugging failures
 - redundancy in generated tests
 - strengths and limitations of combinatorial testing inputs
-

8. Mutation Testing Requirements

You must use PIT mutation testing.

You should report:

- mutation score
- killed mutants
- surviving mutants
- timed-out mutants
- equivalent-mutant discussion

You must analyze at least 5 surviving mutants and discuss:

- why they survived
 - whether additional tests could kill them
 - whether some appear equivalent
-

9. Combinatorial Interaction Testing Using PICT

A central requirement of this assignment is the use of combinatorial interaction testing (CIT).

You must model relevant input spaces and use Microsoft PICT to generate test inputs.

PICT repository: <https://github.com/microsoft/pict>

9.1 Required Combinatorial Coverage

You must generate test inputs with 2-wise, 3-wise, and 4-wise combinatorial coverage.

You may generate higher-strength combinations if desired.

9.2 Input-Space Modeling

You must design one or more PICT models representing meaningful automata and regex input characteristics.

Possible parameters include:

- regex length
- character set
- quantifier type
- anchors
- alternation depth
- nesting depth
- input-string length
- automaton operation

You are encouraged to justify parameter choices based on expected sources of faults.

9.3 Integration with Testing

PICT-generated combinations must be integrated into your testing infrastructure.

For example:

- parameterized JUnit5 tests
 - generated regex/input pairs
 - generated automata-operation sequences
 - generated metamorphic-test inputs
-

9.4 Evaluation of Combinatorial Testing

You must discuss:

- how the input model was designed
- why specific parameters were chosen
- how many combinations were generated
- tradeoffs between coverage strength and test-suite size
- strengths and limitations of combinatorial testing
- whether combinatorial inputs exposed interesting bugs or mutants

You must also evaluate the effect of increasing interaction strength.

At minimum, you should compare:

- 2-wise coverage
- 3-wise coverage
- 4-wise coverage

For each interaction strength, report metrics such as:

- number of generated inputs
- mutation score
- additional mutants killed
- test execution time
- coverage metrics

You should discuss:

- whether higher interaction strength improves fault detection
- diminishing returns of higher strengths
- computational and practical costs
- scalability concerns

10. Correctness Evaluation Using Metamorphic Testing

You must design and implement metamorphic tests.

You must define at least 10 metamorphic properties.

Examples include:

- union commutativity
- double complement
- idempotent minimization
- determinization preservation
- intersection subset relation
- concatenation associativity
- regex roundtrip consistency

You must:

- implement automated metamorphic tests
- explain why each property should hold
- identify violations if found
- discuss oracle limitations

You are encouraged to combine:

- random generation
- combinatorial interaction testing
- metamorphic transformations

to systematically explore the automata input space.

11. Differential Testing Against `java.util.regex`

You must compare selected BRICS behaviors against `java.util.regex`.

The purpose is to:

- identify semantic differences
- validate correctness assumptions
- expose edge cases
- complement metamorphic and combinatorial testing

You should identify:

- disagreements
 - unsupported regex constructs
 - semantic differences
 - possible bugs
-

12. Software-Design Evaluation

You must evaluate the software-design quality of the BRICS automata library.

You may use tools such as:

- SpotBugs
- Checkstyle
- SonarQube
- IntelliJ inspections

However, the assignment also requires substantial manual or AI-assisted software-design analysis.

12.1 Required Design-Analysis Topics

You must analyze the library with respect to:

- SOLID principles
 - design patterns
 - API design
 - maintainability
 - extensibility
 - testability
-

12.2 SOLID Evaluation

You must evaluate whether important parts of the library adhere to or violate the SOLID principles.

You should provide concrete examples from the codebase.

12.3 Design-Pattern Analysis

You must identify and discuss important design patterns used in the library.

Examples may include:

- Factory
- Builder
- Strategy
- Iterator
- Singleton
- Composite

For each identified pattern, discuss:

- where it appears
 - why it may have been used
 - advantages
 - disadvantages
 - possible alternatives
-

12.4 Refactoring Proposals

You must propose at least 3 concrete refactoring suggestions.

Each proposal must include:

- affected classes or methods
 - description of the problem
 - proposed refactoring
 - expected benefits
 - possible risks or tradeoffs
-

13. Validation of Your Own Infrastructure

You are building experimental infrastructure and must validate it.

You must write tests for your own code.

At minimum test:

- metamorphic-test generators

- mutation-analysis scripts
- result parsers
- coverage aggregation code
- HTML-generation scripts
- combinatorial-test generation scripts

You should measure coverage for your infrastructure code.

14. Automation Requirements

Your artifact should automate:

- building
 - test execution
 - PIT execution
 - coverage collection
 - metamorphic testing
 - combinatorial testing
 - differential testing
 - report generation
 - website generation
 - PICT model execution
 - generation of 3-wise combinations
 - conversion of combinations into JUnit5 inputs
-

15. HTML Website Requirements

Your final report must be a professional looking static HTML website.

The website should include the following sections:

- Overview
- Generated JUnit5 Test Suite
- Combinatorial Interaction Testing
- Metamorphic Testing
- Differential Testing
- Software-Design Evaluation
- Threats to Validity
- Reproducibility
- Use of AI Tools

The Combinatorial Interaction Testing section must include:

- PICT models
- parameter-selection rationale

- comparison of different interaction strengths
 - mutation-score comparison across strengths
 - growth in generated test-suite size
 - analysis of diminishing returns
 - discussion of cost versus effectiveness
-

16. Reproducibility Artifact

Your final artifact must include:

- source code
 - generated tests
 - metamorphic tests
 - PICT models
 - generated 3-wise combinations
 - generated combinatorial tests
 - PIT reports
 - coverage reports
 - static-analysis reports
 - software-design analysis report
 - SOLID evaluation notes
 - refactoring proposals
 - scripts
 - website
 - README
-

17. Project Presentation Video

In addition to the website and reproducibility artifact, you must submit a short project presentation video. The purpose of the video is to provide a concise overview of your work, methodology, results, and conclusions.

17.1 Requirements

The video must:

- be between **5 and 10 minutes** long
- use presentation slides in English only
- include screenshots from different parts of your project
- include narration by the student(s) in English or Hebrew
- not show the student(s) on camera

- be submitted in a common video format such as MP4

The video should consist primarily of:

- slides
- screenshots
- figures
- tables
- plots
- diagrams

The student's voice must be recorded as a voiceover throughout the presentation.

17.2 Suggested Structure

Introduction

- project goals
- overview of the BRICS library
- overview of the two main goals of the assignment

Test Generation and Mutation Testing

- generated JUnit5 test suite
- coverage results
- PIT mutation-testing results

Combinatorial Interaction Testing

- PICT model
- selected parameters
- generated combinations
- comparison of interaction strengths
- mutation-score results

Correctness Evaluation

- metamorphic properties
- differential testing
- interesting findings

Software-Design Evaluation

- SOLID analysis

- identified design patterns
- selected refactoring proposals

Conclusions

- key findings
 - lessons learned
 - limitations
 - future improvements
-

17.3 Required Screenshots

The video must include screenshots from several parts of the project, such as:

- repository structure
- generated JUnit tests
- PICT models
- generated combinations
- PIT reports
- coverage reports
- metamorphic tests
- differential-testing results
- static-analysis results
- HTML website
- selected source-code examples
- selected refactoring examples

You are encouraged to annotate screenshots to help explain important observations.

17.4 Evaluation Criteria

The video will be evaluated based on:

- clarity of presentation
- technical accuracy
- quality of explanation
- effective use of slides and screenshots
- discussion of results
- organization and pacing
- professionalism

18. Weekly Milestones

Week 1 — Setup and Initial Planning

Deliverables:

- repository setup
 - successful build
 - baseline coverage measurement
 - initial metamorphic-property list
 - initial PICT input-space model draft
 - initial parameter-selection rationale
 - repository skeleton
 - README draft
-

Week 2 — Test Generation and Combinatorial Testing

Deliverables:

- first generated JUnit5 suite
 - first working PICT models
 - generated 3-wise combinations
 - integration of PICT-generated inputs into JUnit5 tests
 - metamorphic-testing framework
 - at least 5 implemented metamorphic properties
 - PIT execution results
 - infrastructure tests
 - initial comparison between 2-wise and 3-wise combinations
 - preliminary mutation-score comparison across interaction strengths
-

Week 3 — Correctness and Design Evaluation

Deliverables:

- complete metamorphic suite
- differential-testing results
- static-analysis results
- initial SOLID-principles evaluation
- identification of important design patterns
- draft refactoring proposals
- evaluation of the effect of interaction strength on mutation score

- analysis of growth in generated test-suite size
 - discussion of tradeoffs between effectiveness and cost
 - preliminary plots and tables
-

Week 4 — Final Artifact and Website

Deliverables:

- complete website
 - final reproducibility artifact
 - final reports
 - final plots
 - final analysis
 - cleaned repository
 - finalized software-design analysis
 - finalized refactoring proposals with justification
 - discussion of tradeoffs and risks
 - complete documentation
-

20. Grading Rubric

Generated JUnit5 test suite	20	
Mutation testing and PIT analysis	15	
Combinatorial interaction testing with PICT, including interaction-strength evaluation		25
Metamorphic testing	15	
Differential testing	5	
Code-quality and software-design evaluation		10
Reproducibility artifact	5	
HTML website quality	5	
Video presentation quality	5	
Infrastructure validation	3	
Threats to validity and AI-tool discussion	2	

21. Use of AI-Based Tools

You are encouraged to use AI-based tools during this assignment.

You may use them for:

- generating tests
- writing scripts
- debugging
- improving explanations

- creating plots
- improving website quality
- helping design combinatorial models

However, you remain responsible for correctness.

Your final website must include a section titled “Use of AI Tools” explaining:

- which tools were used
 - what they were used for
 - how outputs were validated
 - what mistakes were discovered and corrected
-

22. Submission Requirements

Submit:

1. Source-code repository
2. Reproducibility artifact
3. Static HTML website
4. Project presentation video (5–10 minutes)

Do not submit API keys or private credentials.

Your repository must include clear reproduction instructions.

23. Final Notes

This assignment is intended to simulate a realistic empirical software engineering project involving:

- testing research
- software quality evaluation
- experimental methodology
- reproducibility
- software design analysis

You are encouraged to:

- automate as much as possible
- justify design decisions
- critically analyze your results
- discuss limitations honestly